面向展厅展示场景的工程落实步骤

1. 项目目标与官方约束

本项目围绕“面向展厅展示场景的多模态 AI 交互终端”展开，目标是做出一个可演示、可联调、可继续扩展的完整原型。系统由手持终端、网页展示中控、云端大模型服务三部分组成，重点体现多模态交互、多屏协同、传感器融合与低功耗管理。

根据乐鑫 2026 年全国大学生物联网设计竞赛官网，本项目在工程落地时必须满足以下硬约束：

- 核心主控必须使用 `ESP32-S3`、`ESP32-C5`、`ESP32-P4` 之一。
- 至少实现 1 种传感器数据融合。
- 至少接入 1 个云端大模型服务。
- 设备需要具备上行或下行交互能力。
- 推荐开发环境为 `ESP-IDF v5.5.2`。

结合现有技术路线，本项目默认采用 `ESP32-C5` 路线，并优先围绕官网推荐的 `ESP-SensairShuttle` 套件推进。

2. 开发环境与仓库基线

2.1 Python 运行约束

后续所有 Python 操作都必须在当前仓库根目录下的 `.venv` 中执行，不允许直接调用系统 Python。标准流程固定如下：

```
cd D:\Pass1on_W\Project\zhizhanSEU
.\.venv\Scripts\Activate.ps1
python -m pip install <package>
python <script.py>
```

如果 PowerShell 首次激活被系统策略拦截，可在当前用户范围执行一次：

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

如果 `.venv` 丢失或损坏，使用以下方式重新创建：

```
cd D:\Pass1on_W\Project\zhizhanSEU
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

2.2 ESP-IDF 环境

嵌入式主线开发环境固定为 `ESP-IDF v5.5.2`。Windows 下建议优先使用乐鑫官方一键安装工具完成环境部署，并以 `idf.py` 作为主构建入口。工程落地阶段统一遵循以下约定：

- 端侧固件主工程使用 `ESP-IDF v5.5.2`。
- 端侧目标芯片固定为 `esp32c5`。
- 所有驱动、组件和示例优先选用官网列出的官方资源。
- 如需国内镜像，优先使用乐鑫官网说明中的极狐镜像。

建议在端侧工程建立后使用以下命令完成基础验证：

```
idf.py set-target esp32c5
idf.py build
idf.py flash
idf.py monitor
```

建议尽早建立固定目录约定，避免后续文件散落。推荐最小目录分工如下：

- `firmware/`：ESP-IDF 端侧工程
- `web/`：网页中控前端工程
- `docs/`：补充说明、演示脚本、接口草案
- 仓库根目录保留竞赛原始材料与最终整理文档

在真正开始编码前，建议先完成三个“环境是否可用”的快速判断：

- `.venv` 激活后能正常运行 Python
- `idf.py --version` 能返回 `ESP-IDF v5.5.2`
- 设备接入电脑后能识别出稳定串口

2.3 前端双 skill 工作流

本项目后续所有网站或展示页面工作，统一采用双 skill 协作机制：

1. `frontend-design` 负责快速生成高美感视觉方向、示例页面、风格样张和界面 archetype。
2. `frontend-skill` 负责将确认后的方向落地为可运行、可维护、可演示的网页代码。

3. 如果两者输出存在冲突，以 `frontend-design` 的视觉方向作为美术依据，以 `frontend-skill` 的实现结果作为最终工程交付依据。

该流程的目的不是做两套页面，而是先解决审美方向，再解决代码交付。

3. 板卡与硬件 bring-up

3.1 主线硬件选型

本项目默认使用官网“套件一” `ESP-SensairShuttle`。该套件与当前技术路线高度一致，核心资源包括：

- `ESP32-C5-WROOM-1-N16R8`
- `BME690`
- `BMI270 & BMM350`
- `1.83` 寸 LCD
- 麦克风
- 扬声器接口
- `I2C`、RGB 灯带、外置引脚等扩展接口

该套件适合直接支撑以下核心能力：

- 姿态检测与动作识别
- 温度、湿度、气压、空气质量等环境信息采集
- 语音输入与 AI 对话
- 屏幕本地信息显示
- 云端大模型上下行交互
- 多种外设扩展和二次开发

3.2 硬件 bring-up 顺序

硬件 bring-up 不应一开始就追求完整功能，而应按最小闭环拆解：

1. 串口、烧录与基础固件启动验证。
2. LCD 点亮与简单文本显示。
3. 麦克风输入链路打通。
4. 扬声器输出链路打通。
5. `BMI270` 与 `BMM350` 的基础读取。
6. `BME690` 的基础读取。
7. Wi-Fi 联网与配网流程验证。

每一步都要产出一个能复验的小结果，而不是只看串口日志。建议阶段性产物如下：

- LCD 阶段：屏幕上显示固定文本与设备状态
- 麦克风阶段：确认能采到有效音频缓冲
- 扬声器阶段：播放固定提示音
- IMU 阶段：串口输出稳定姿态状态
- `BME690` 阶段：串口或网页中能看到温度、湿度、气压、空气质量等环境数据
- 网络阶段：页面或日志中显示联网成功与 IP 信息

3.3 传感器融合主线

本项目的最低可行融合方案固定为 `BMI270 + BMM350`，用来实现姿态和方向相关融合判断，驱动终端状态机。建议最先落地以下状态：

- 拿起唤醒
- 静置待机
- 交互中保持唤醒
- 指定动作触发展项切换或讲解开始

`BME690` 作为扩展感知能力保留，后续可用于环境状态驱动展项切换或场景联动，但不进入第一阶段最小闭环。同时，`BME690` 提供的温度、湿度、气压和空气质量数据可作为网页中控的扩展展示字段，在无板阶段先用模拟值占位，待实物板接入后再切换为真实采样结果。

为避免前期算法投入过重，第一版融合逻辑建议采用“规则优先”策略，而不是一开始就做复杂模型：

- 先用阈值、方向区间、持续时间做基本状态判定
- 等演示闭环稳定后，再考虑优化灵敏度和误触发率

4. 端侧软件拆分与实现顺序

端侧软件建议按模块拆分，避免把所有逻辑堆进单一主循环。建议最小模块如下：

- `audio_input`：音频采集、缓存与上传前处理
- `audio_output`：播报、提示音、返回语音播放
- `imu_fusion`：姿态与方向融合判断
- `state_machine`：待机、激活、交互、休眠四态切换
- `networking`：Wi-Fi、配网、基础连接管理
- `ws_client`：与网页中控的实时消息同步
- `cloud_bridge`：与云端对话智能体的数据收发桥接
- `display_ui`：终端摘要显示、状态提示、菜单导航

建议实现顺序如下：

1. `networking` + `display_ui`，先完成联网和最基本屏显。
2. `imu_fusion` + `state_machine`，完成拿起唤醒与静置待机。
3. `audio_input`，完成端侧采音上送。
4. `cloud_bridge`，完成与云端智能体的对接。
5. `ws_client`，完成网页大屏联动。
6. `audio_output`，补齐完整讲解与提示反馈。

建议每完成一个模块就做一次局部验收，不把多个关键链路压到同一天联调。最小局部验收标准如下：

- `networking`：设备上电后能稳定联网，并且断线后可恢复
- `display_ui`：终端能显示当前状态、网络状态与最近一次操作
- `imu_fusion`：拿起与静置动作能稳定区分
- `cloud_bridge`：发送一次请求后，能收到结构化返回结果
- `ws_client`：终端一次动作能稳定触发网页一次变化
- `audio_output`：收到云端结果后能播报或至少发出提示反馈

如果进度紧张，最小可行原型的优先级固定如下：

1. 状态机可用
2. 云端问答可用
3. 网页联动可用
4. 本地屏显可用
5. 语音播报与高级 UI 优化

也就是说，优先保证“能演示”，再优化“演示得更漂亮”。

5. 云端大模型接入与配置

5.1 主方案

云端主方案固定采用官网提供的火山引擎“硬件对话智能体”。原因如下：

- 官网明确提供专项资源与产品入口。
- 与硬件语音链路适配度高，适合“会听、会说”的设备场景。
- 官方提供 `GitHub` 代码仓、产品文档和 license 支持。
- 比通用文本 API 更接近本项目的语音交互原型需求。

5.2 云端接入步骤

建议按如下顺序推进：

1. 注册火山账号并完成实名认证。
2. 开通“硬件对话智能体”产品。
3. 获取基础版本 license。
4. 阅读官方产品文档和官方代码仓，确认设备接入流程。
5. 确定设备标识、会话模式、语音上传方式和返回数据格式。
6. 把返回结果拆成两类输出：
    - 终端摘要
    - 网页完整展示内容

建议云端返回结果在本项目内统一整理为一个中间结构，至少包含以下字段：

- `intent`：当前问题或当前展项对应的任务类型
- `title`：网页主标题或展项名称
- `summary`：终端侧摘要
- `detail`：网页侧完整讲解内容
- `followup`：可选追问或下一步操作提示

这样可以减少端侧和网页端对云端原始数据格式的强耦合。

建议网页与端侧之间的实时消息也采用统一结构，至少包含：

- `event`：例如 `wake`、`select_exhibit`、`ask_question`、`cloud_result`
- `timestamp`
- `device_state`
- `payload`

这样可以让联调时更容易定位问题到底出在端侧、云端还是网页端。

5.3 备选方案

如果主方案推进受阻，可使用以下方案作为备选：

- 火山边缘大模型网关
- 百度文心 API
- 官网列出的其他云端大模型官方接入教程

但备选方案只用于兜底，不作为主文档实施顺序。

6. 网页中控与前端工作流

6.1 网页中控角色边界

网页中控不是自由创作的单页站点，而是赛题原型的一部分。它的职责固定为：

- 展项切换
- 多屏联动
- 问答结果展示
- 演示状态提示
- 终端控制事件映射

因此前端实现必须以“展示控制台”思路为主，而不是营销页思路。

6.2 前端结构建议

网页中控建议按以下区域组织：

- 主展示区：展项大图、讲解正文、扩展资料
- 当前状态区：设备连接、当前模式、语音状态、云端状态，以及温度、湿度、气压、空气质量等环境数据
- 事件反馈区：最近一次动作触发、最近一次问答、最近一次切页
- 演示控制区：手动切换展项、演示模式开关、联调按钮

前端实现时应优先保证以下三个目标：

- 大屏远距离可读
- 状态切换有明显反馈
- 与终端动作之间存在一一对应的视觉变化

前端实现阶段不建议一开始就追求复杂组件库。应优先把页面做成“单一演示工作面”，确保：

- 第一屏就能看懂当前展项和当前状态
- 用户发起操作后，网页有立刻可见的反馈
- 讲解内容与终端摘要之间语义一致
- 无板阶段的环境数据必须明确标记为模拟值，不写成真实板级采样

6.3 前端双 skill 使用方式

实际工作时严格按以下顺序执行：

1. 先用 `frontend-design` 产出 1 至 2 个高质量视觉方案。
2. 从中选定最适合展厅大屏的视觉方向。
3. 再用 `frontend-skill` 将该方向实现为可运行页面。

4. 页面实现必须保留：
    - 大屏可读性
    - 状态提示清晰度
    - 终端事件到网页变化的直观映射

建议把前端产物拆成两类：

- 视觉参考稿：由 `frontend-design` 给出，主要解决风格、版式、色彩和动效方向
- 工程实现稿：由 `frontend-skill` 产出，主要解决响应式、组件结构、状态流和实际演示效果

7. 联调顺序与阶段验收

7.1 联调顺序

联调时不要同时拉太多链路，建议固定顺序如下：

1. 终端基础启动正常。
2. Wi-Fi 连通正常。
3. 传感器状态切换正常。
4. 语音采集正常。
5. 云端对话返回正常。
6. 网页联动正常。
7. 待机与唤醒正常。
8. 完整演示流程正常。

7.2 阶段验收标准

建议分五个阶段验收：

- 阶段 1：环境完成
    - `.venv` 可用
    - `ESP-IDF v5.5.2` 可编译
- 阶段 2：硬件通路完成
    - 屏幕、麦克风、IMU、网络至少完成基础验证
- 阶段 3：云端对话可用
    - 设备能把输入送到火山智能体，并收到有效返回
- 阶段 4：网页联动可用
    - 终端操作能触发网页状态变化
- 阶段 5：比赛闭环可演示

    - 用户拿起 -> 提问/选择 -> 网页变化 -> 返回摘要 -> 设备待机

每个阶段都应保留一个可演示截图、串口片段或短视频，后续既可用于内部复盘，也可用于答辩材料整理。

联调阶段最常见的阻塞点通常来自以下几类，建议提前安排排查顺序：

- 设备未稳定联网
- 音频采集链路有噪声或采样格式不匹配
- 云端 license、鉴权或配置未完成
- WebSocket 消息结构不统一
- 页面已经切换，但内容字段映射错误

8. 比赛演示闭环与里程碑

8.1 最小可行演示闭环

比赛演示应优先保证“完整闭环”，而不是一次堆太多功能。推荐闭环如下：

1. 用户拿起终端。
2. 设备由 IMU 融合结果触发唤醒。
3. 用户通过触摸选择展项，或通过语音发起提问。
4. 设备将事件同步到网页中控，并将语音上送云端。
5. 云端返回讲解结果。
6. 终端显示摘要，网页大屏显示完整讲解内容。
7. 用户放下终端，设备进入待机。

如果比赛前时间不足，演示脚本必须优先保住这个闭环，不再临时增加新特性。任何新增特性都只能建立在不破坏该闭环的前提下。

8.2 里程碑规划

- M1：开发环境与仓库规范完成
- M2：硬件 bring-up 完成
- M3：姿态唤醒与低功耗状态机完成
- M4：云端对话链路完成
- M5：网页中控与 WebSocket 联动完成
- M6：完整比赛演示闭环完成

建议把每个里程碑都和明确交付物绑定：

- M1：`.venv`、ESP-IDF 环境、前端 skill 全部可用

- M2：端侧外设自检通过
- M3：状态机演示通过
- M4：一次完整语音问答成功
- M5：终端与网页双端同步成功
- M6：完整讲解演示成功

建议再补一个答辩准备动作：在 M6 完成后，用同一套演示流程反向整理竞赛展示材料，包括：

- 系统架构图
- 模块划分图
- 演示流程图
- 实物照片与网页展示截图

9. 官方资源清单

工程实施时优先参考以下官方资源方向：

- 乐鑫竞赛官网
- `ESP-IDF v5.5.2`
- `ESP-SensairShuttle` 套件说明
- 火山硬件对话智能体产品文档与官方代码仓
- `ESP-SR`
- `AFE`
- `ESP-ADF`
- `Provisioning`
- `ESP-Spot`

这些资源共同构成当前项目最可靠的官方落地路径。

建议在实际执行时，把以下两个页面作为外部事实依据长期保留：

- 赛题官网：https://www.espressif.com/zh-hans/ecosystem/education/competition/iot
- 当前上游技术说明：技术路线.md